

Student Management System Using Queue Linked List and CSV File Handling in C++

Name	ID
HIN PANHA	e20241363
HOR SOUE HAK	e20240263
KUN VIRAK	e20240486

Group 6 : I2-AMS-1B

1. Libraries and Structure Definition

- **Code**

```
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

using namespace std;

struct Student {
    string name;
    string id;
    int age;
    string gender;
    string department;
    string phone;
    string birth;
    string email;
    int year;
    string major;
    string province;

    Student* next;
};
```

- Import required libraries.
- Define the Student structure.
- Store student information such as ID, name, age, department, phone number, and email.
- Use a pointer next to create a linked list.

2. Queue Initialization

- **Code**

```
Student* front = NULL;
Student* rear = NULL;
```

- Initialize the queue.
- front points to the first student.
- rear points to the last student.
- Used for managing student records dynamically.

3. Save Data to File

- Code

```
//Save data to CSV file
void SaveFile () {
    ofstream file("Students_Information.csv");
    file << "ID,Name,Age,Gender,Department,Major,Year,Province,Phone,Birth,Email" << endl;

    Student* current = front;
    while (current != NULL) {
        file << current -> id << ","
            << current -> name << ","
            << current -> age << ","
            << current -> gender << ","
            << current -> department << ","
            << current -> major << ","
            << current -> year << ","
            << current -> province << ","
            << current -> phone << ","
            << current -> birth << ","
            << current -> email << endl;

        current = current -> next;
    }
    file.close();
}
```

- Save student records into a CSV file.
- Allow data persistence after the program closes.
- Write all information from the queue to Students_Information.csv.

4. Load Data from File

- Code

```
//Turn data from CSV file into queue
void DataIntoQueue () {
    ifstream file("Students_Information.csv");
    if (!file) {
        return;
    }

    string skip;
    getline (file, skip); //to skip header

    string id, name, ageText, gender, department, major, yearText, province, phone, birth, email;

    while (getline (file, id, ',') && //read data comma by comma
        getline (file, name, ',') &&
        getline (file, ageText, ',') &&
        getline (file, gender, ',') &&
        getline (file, department, ',') &&
        getline (file, major, ',') &&
        getline (file, yearText, ',') &&
        getline (file, province, ',') &&
        getline (file, phone, ',') &&
        getline (file, birth, ',') &&
        getline (file, email)) {
```

```

        Student* newStudent = new Student();

        //store whole data into queue
        newStudent -> id = id;
        newStudent -> name = name;
        newStudent -> age = stoi(ageText); //stoi: turn string to integer
        newStudent -> gender = gender;
        newStudent -> department = department;
        newStudent -> major = major;
        newStudent -> year = stoi(yearText);
        newStudent -> province = province;
        newStudent -> phone = phone;
        newStudent -> birth = birth;
        newStudent -> email = email;

        newStudent -> next = NULL;

        if (rear == NULL){
            front = rear = newStudent;
        }else {
            rear -> next = newStudent;
            rear = newStudent;
        }
    }
    file.close();
}
}

```

- Read student data from the CSV file.
- Convert file data into queue nodes.
- Restore previously saved records when the program starts.

5. Add Student

- Code

```

//Add Student
void addStudent () {
    Student* newStudent = new Student;
    Student* idStudent = front;

    cout << "\nEnter Student ID: "; cin >> newStudent->id;

    while (idStudent != NULL){
        if (newStudent->id == idStudent->id){
            cout << "\nThis student's ID is already registered" << endl;
            return;
        }
        idStudent = idStudent->next;
    }

    cin.ignore();
    cout << "Enter Name: "; getline(cin, newStudent -> name);

    cout << "Enter Age: "; cin >> newStudent -> age;
    cout << "Enter Gender: "; cin >> newStudent -> gender;
    cout << "Enter Department: "; cin >> newStudent -> department;

    cin.ignore();
    cout << "Enter Major: "; getline(cin, newStudent -> major);

    cout << "Enter Year: "; cin >> newStudent -> year;
    cout << "Enter Province: "; cin >> newStudent -> province;
}

```

```

    cin.ignore();
    cout << "Enter Phone Number: "; getline(cin, newStudent -> phone);

    cout << "Enter Birth (DD/MM/YY): "; cin >> newStudent -> birth;
    cout << "Enter Email: ";    cin >> newStudent -> email;

    newStudent -> next = NULL;

    if (rear == NULL){
        front = rear = newStudent;
    }else { //update rear
        rear -> next = newStudent;
        rear = newStudent;
    }
    SaveFile();
    cout << "\nStudent Added Successfully." << endl;
}

```

- **Result**

```

Enter Student ID: 123
Enter Name: rak
Enter Age: 18
Enter Gender: male
Enter Department: ams
Enter Major: data science
Enter Year: 2
Enter Province: kompong cham
Enter Phone Number: 1234567
Enter Birth (DD/MM/YY): 12.08.05
Enter Email: rak@123

Student Added Successfully.

```

6. Remove Student

- **Code**

```

//Remove student
void removeStudent () {
    if (front == NULL) {
        cout << "\nThere is no student to show..!" << endl;
        return;
    }

    string targetID;
    cout << "\nEnter Student's ID to delete: ";    cin >> targetID;

    Student* current = front; //current point to first node
    Student* prev = NULL;    //prev point to NULL

    while (current != NULL && current -> id != targetID) {
        prev = current;
        current = current -> next;
    }

    if (current == NULL) {
        cout << "\nStudent not found" << endl;
        return;
    }

    if (prev == NULL) {

```

```

if (prev == NULL){
    front = current -> next; //update the second node to be the first and the first removed
}else {
    prev -> next = current -> next;
}

if (current == rear){
    rear = prev;
}

delete current;
SaveFile();

cout << "\nStudent delete successfully." << endl;
}

```

- Before

Student List										
ID	Name	Age	Gender	department	Major	Year	Province	Phone	Birth	Email
234	nha	18	male	ams	computer science	2	battambang	97876543	30.08.06	nha@345
123	rak	18	male	ams	data science	2	kompong cham	1234567	12.08.05	rak@123

- After delete

Student Management System										
1. Add Student 2. Remove Student 3. Display Students 4. Search Student 5. Update Student 6. Exit										
Enter Choice: 2										
Enter Student's ID to delete: 123										
Student delete successfully.										
Student Management System										
1. Add Student 2. Remove Student 3. Display Students 4. Search Student 5. Update Student 6. Exit										
Enter Choice: 3										
Student List										
ID	Name	Age	Gender	department	Major	Year	Province	Phone	Birth	Email
234	nha	18	male	ams	computer science	2	battambang	97876543	30.08.06	nha@345

- Delete a student by ID.
- Search for the target record.
- Update queue links after deletion.
- Save changes to the file.

7. Sort Student Records

- Code

```
//Sorted name A -> Z
void sortName() {
    if (front == NULL || front -> next == NULL)
        return;

    bool swapped;
    Student* current;

    do {
        swapped = false;
        current = front;

        while (current -> next != NULL) {
            if (current -> name > current -> next -> name) {

                swap(current -> id, current -> next -> id);
                swap(current -> name, current -> next -> name);
                swap(current -> age, current -> next -> age);
                swap(current -> gender, current -> next -> gender);
                swap(current -> department, current -> next -> department);
                swap(current -> major, current -> next -> major);
                swap(current -> year, current -> next -> year);
                swap(current -> province, current -> next -> province);
                swap(current -> phone, current -> next -> phone);
                swap(current -> birth, current -> next -> birth);
                swap(current -> email, current -> next -> email);

                swapped = true;
            }

            current = current -> next;
        }
    } while (swapped);
}
```

- Sort students alphabetically by name.
- Use Bubble Sort technique.
- Improve readability when displaying records.

8. Display Student

- Code

```

//unspacy student
void displayStudent () {
    if (front == NULL) {
        cout << "\nThere is no student..." << endl;
        return;
    }
    sortName();
    //header
    cout << "\n\n---- Student List ----\n\n";
    cout << left
        << setw(15) << "ID"
        << setw(20) << "Name"
        << setw(10) << "Age"
        << setw(15) << "Gender"
        << setw(15) << "Department"
        << setw(20) << "Major"
        << setw(10) << "Year"
        << setw(15) << "Province"
        << setw(20) << "Phone"
        << setw(20) << "Birth"
        << setw(30) << "Email" << endl;

    Student* show = front;
    while (show != NULL) {
        cout << left
            << setw(15) << show->id
            << setw(20) << show->name
            << setw(10) << show->age
            << setw(15) << show->gender
            << setw(15) << show->department
            << setw(20) << show->major
            << setw(10) << show->year
            << setw(15) << show->province
            << setw(20) << show->phone
            << setw(20) << show->birth
            << setw(30) << show->email << endl;

        show = show->next;
    }
}

```

- Display all student records.
- Show information in table format using setw().
- Call the sorting function before displaying.

9. Search Student

- Code

```

Student* findStudent (string id) {
    Student* look = front; //look point to the first node

    while (look != NULL) {
        if (look->id == id) {
            return look;
        }

        look = look->next;
    }
    return NULL;
}

//Search student
void searchStudent () {
    if (front == NULL) {
        cout << "\nThere is no student." << endl;
        return;
    }

    string targetID;
    cout << "\nEnter ID student to see their information: ";    cin >> targetID;

    Student* found = findStudent(targetID);

    if (found == NULL) {
        cout << "\nStudent not found." << endl;
        return;
    }
}

```

```

cout << "\nStudent Found!\n";
cout << "ID: " << found -> id << endl;
cout << "Name: " << found -> name << endl;
cout << "Age: " << found -> age << endl;
cout << "Gender: " << found -> gender << endl;
cout << "Department: " << found -> department << endl;
cout << "Major: " << found -> major << endl;
cout << "Year: " << found -> year << endl;
cout << "Province: " << found -> province << endl;
cout << "Phone: " << found -> phone << endl;
cout << "Birth: " << found -> birth << endl;
cout << "Email: " << found -> email << endl;
}

```

- **Result**

```

----- Student Management System -----
1. Add Student
2. Remove Student
3. Display Students
4. Search Student
5. Update Student
6. Exit

Enter Choice: 4

Enter ID student to see their information: 777

Student Found!
ID: 777
Name: rak
Age: 19
Gender: male
Department: ams
Major: data science
Year: 2
Province: phnom penh
Phone: 02223223
Birth: 22.03.06
Email: rak@123

```

- Search a student using their ID.
- Display complete student information if found.
- Inform the user if the record does not exist.

10. Update Student Information

- **Code**

```

//Update student's information by their ID
void updateStudent () {
    if (front == NULL) {
        cout << "\nThere is no student." << endl;
        return;
    }

    string targetID;
    cout << "\nEnter ID student to update their information: ";    cin >> targetID;

    Student* update = findStudent(targetID);

    if (update == NULL) {
        cout << "\nStudent not found." << endl;
        return;
    }

    int choice;
    cout << "\n----- Update Menu ----- \n";
    cout << "\n01. Update ID\n";
    cout << "02. Update Name\n";
    cout << "03. Update Age\n";
    cout << "04. Update Gender\n";
    cout << "05. Update Department\n";
    cout << "06. Update Major\n";
    cout << "07. Update Year\n";
    cout << "08. Update Province\n";

    cout << "09. Update Phone\n";
    cout << "10. Update Birth\n";
    cout << "11. Update Email\n";

    cout << "\nEnter Choice: "; cin >> choice;

    switch (choice) {
        case 1:
            cout << "Enter New ID: ";
            cin >> update->id;
            break;

        case 2:
            cin.ignore();
            cout << "Enter New Name: ";
            getline(cin, update->name);
            break;

        case 3:
            cout << "Enter New Age: ";
            cin >> update -> age;
            break;

        case 4:
            cout << "Enter New Gender: ";
            cin >> update -> gender;
            break;
    }
}

```

```

        case 5:
            cout << "Enter New Department: ";
            cin >> update -> department;
            break;

        case 6:
            cin.ignore();
            cout << "Enter New Major: ";
            getline(cin, update -> major);
            break;

        case 7:
            cout << "Enter New Year: ";
            cin >> update -> year;
            break;

        case 8:
            cin.ignore();
            cout << "Enter New Province: ";
            getline(cin, update -> province);
            break;

        case 9:
            cin.ignore();
            cout << "Enter New Phone Number: ";
            getline(cin, update -> phone);
            break;

        case 10:
            cout << "Enter New Birth (DD/MM/YYYY): ";
            cin >> update -> birth;
            break;

        case 11:
            cout << "Enter New Email: ";
            cin >> update -> email;
            break;

        default:
            cout << "\nInvalid Choice.\n";
            return;
    }

    SaveFile();

    cout << "\nStudent information updated successfully.\n";
}

```

- Result

// Change ID

```

----- Student Management System -----

1. Add Student
2. Remove Student
3. Display Students
4. Search Student
5. Update Student
6. Exit

Enter Choice: 5

Enter ID student to update their information: 234

----- Update Menu -----

01. Update ID
02. Update Name
03. Update Age
04. Update Gender
05. Update Department
06. Update Major
07. Update Year
08. Update Province
09. Update Phone
10. Update Birth
11. Update Email

Enter Choice: 1
Enter New ID: 555

Student information updated successfully.

```

- Then

```

----- Student List -----

```

ID	Name	Age	Gender	department	Major	Year	Province	Phone	Birth	Email
987	hak	12	male	gic	computer science	2	kompong cham	028372373	23.01.07	hak@344
555	nha	18	male	ams	computer science	2	battambang	97876543	30.08.06	nha@345
777	rak	19	male	ams	data science	2	phnom penh	02223223	22.03.06	rak@123

//Change by Phone Number

```

----- Student Management System -----

1. Add Student
2. Remove Student
3. Display Students
4. Search Student
5. Update Student
6. Exit

Enter Choice: 5

Enter ID student to update their information: 555

----- Update Menu -----

01. Update ID
02. Update Name
03. Update Age
04. Update Gender
05. Update Department
06. Update Major
07. Update Year
08. Update Province
09. Update Phone
10. Update Birth
11. Update Email

Enter Choice: 9
Enter New Phone Number: 999999999

Student information updated successfully.

```

- Then

```

----- Student List -----

```

ID	Name	Age	Gender	department	Major	Year	Province	Phone	Birth	Email
987	hak	12	male	gic	computer science	2	kompong cham	028372373	23.01.07	hak@344
555	nha	18	male	ams	computer science	2	battambang	999999999	30.08.06	nha@345
777	rak	19	male	ams	data science	2	phnom penh	02223223	22.03.06	rak@123

- Modify existing student information.
- Allow updating specific fields such as name, age, department, phone, and email.
- Save changes automatically.

11. Menu Function

```

// menu
void showMenu() {
    cout << "\n----- Student Management System ----- \n\n";
    cout << "1. Add Student\n";
    cout << "2. Remove Student\n";
    cout << "3. Display Students\n";
    cout << "4. Search Student\n";
    cout << "5. Update Student\n";
    cout << "6. Exit\n";
    cout << "\nEnter Choice: ";
}

int main() {
    DataIntoQueue();

    int choice;

    do {
        showMenu();
        cin >> choice;

        switch (choice) {
            case 1:
                addStudent();
                break;
            case 2:
                removeStudent();
                break;
            case 3:
                displayStudents();
                break;
            case 4:
                searchStudent();
                break;
            case 5:
                updateStudent();
                break;
            case 6:
                cout << "\nProgram Ended.\n";
                break;
            default:
                cout << "\nInvalid Choice!\n";
        }

    } while (choice != 6);

    return 0;
}

```

- **Result**

```

----- Student Management System -----

1. Add Student
2. Remove Student
3. Display Students
4. Search Student
5. Update Student
6. Exit

```

- Display available system options.
- Provide a user-friendly interface.

12. Main Function

- Code

```
int main() {
    DataIntoQueue();

    int choice;

    do {
        showMenu();
        cin >> choice;

        switch (choice) {
            case 1:
                addStudent();
                break;
            case 2:
                removeStudent();
                break;
            case 3:
                displayStudent();
                break;
            case 4:
                searchStudent();
                break;
            case 5:
                updateStudent();
                break;
            case 5:
                updateStudent();
                break;
            case 6:
                cout << "\nProgram Ended.\n";
                break;
            default:
                cout << "\nInvalid Choice!\n";
        }

    } while (choice != 6);

    return 0;
}
```

- Program entry point.
- Load existing data.
- Display menu repeatedly.
- Execute user-selected operations.
- Exit when the user chooses option 6.